

EC-200 DATA STRUCTURES

LECTURE 13

1

EFFICIENCY OF ALGORITHMS

- **Definition of Efficiency:** The Amount of Computer Time and Space that an Algorithm Requires
- We are more interested in **time** complexity. Why?
- Efficiency of algorithms helps comparing different methods of solution for the same problem
- Algorithm Analysis Should be Independent of
 - Specific Implementation languages,
 - Computers and
 - Data
- Efficiency is a concern for large problems (large data size)

WHY TIME COMPLEXITY IS IMPORTANT?

Solution to compute if N is prime?

Program 1	Program 2
<pre>for (int i= 2; i < n ; i++) { if (n % i == 0) { cout<<n<<" is not prime"; break; } }</pre>	<pre>for (int i= 2; i <= √ n ; i++) { if (n % i == 0) { cout<<n<<" is not prime"; break; } }</pre>

Let us assume **1 iteration** takes **1ms** to execute

WHY TIME COMPLEXITY IS IMPORTANT?

n	Program 1 n-1 times	Program 2 $\sqrt{n}$ times
11	10 ms	3.31 ms
101	100 ms	10.04 ms
1033	1032 ms	32.14 ms
2383	2382 ms	48.8 ms
9973	9972 ms	99.8 ms
	• • • $T \propto n$	$T \propto \sqrt{n}$

ALGORITHM GROWTH RATE AS A MEASURE OF EFFICIENCY

- **Growth Rate of an Algorithm:** Measures Algorithm's Time Requirement as a Function of the Problem Size (N)
- Specifies **How Quickly** the Algorithm's **Time Requirement Grows** *as a Function of the Problem Size*
- The Way to Measure a Problem's Size Depends on the Application
 - For a Linked List: Number of Nodes
 - For an array: The Size of Array
 - For a Stack: Number of Items in the Stack
 - Etc.

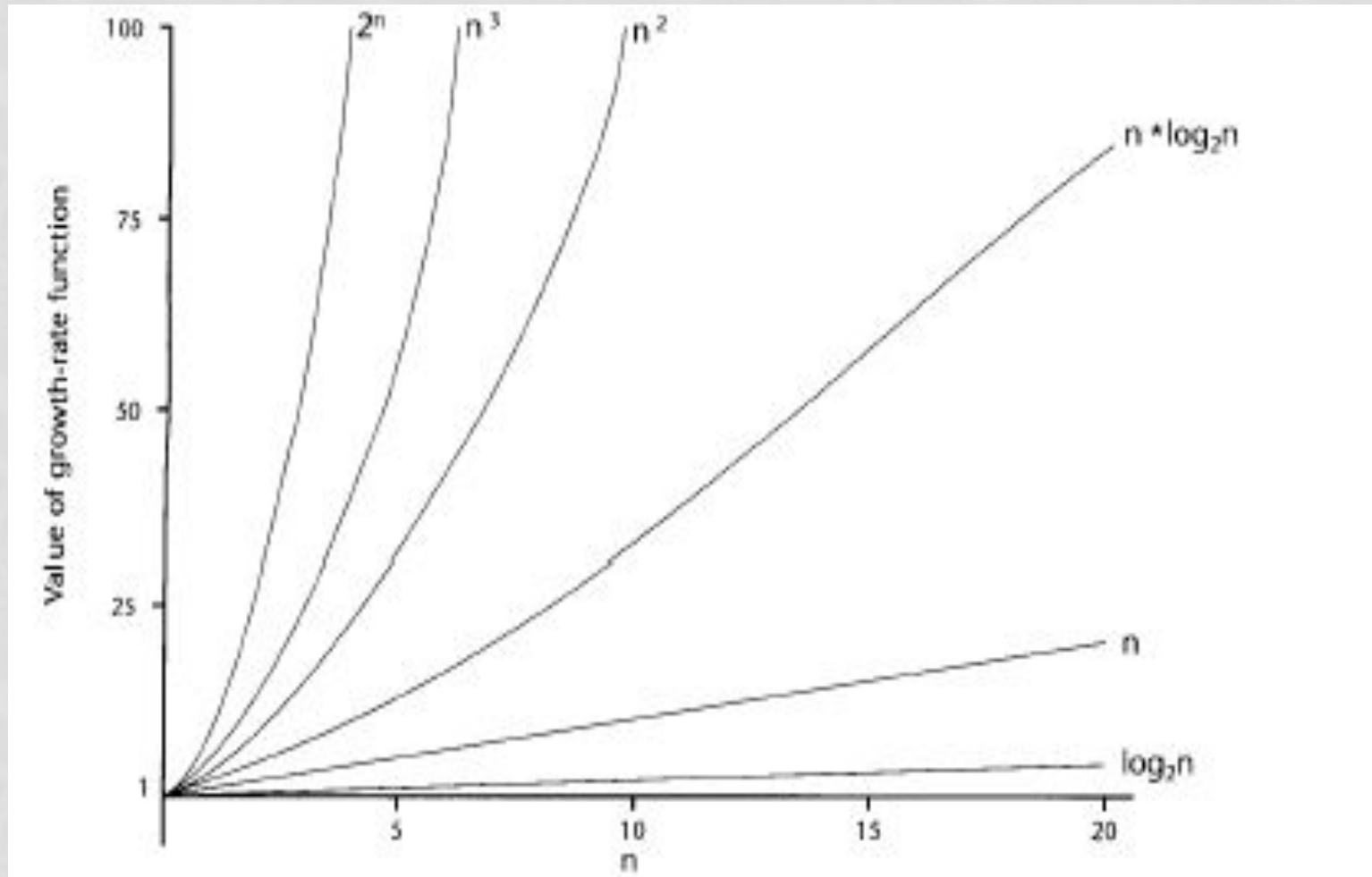
ALGORITHM GROWTH RATES EXAMPLE

- Suppose we have two algorithms A and B for solving the same problem. Which one do we use?
- Suppose

Algorithm A Requires Time Proportional to N^2
Algorithm B Requires Time Proportional to N ,

n	Time increase in A w.r.t. input	Time increase in B w.r.t input
10	10	100
100	100	1000
1000	1000	10000
10000	10000	100000

COMPARISON OF GROWTH RATE



COMPARISON OF GROWTH RATE (TABULAR)

Function	n					
	10	100	1,000	10,000	100,000	1,000,000
1	1	1	1	1	1	1
$\log_2 n$	3	6	9	13	16	19
n	10	10^2	10^3	10^4	10^5	10^6
$n \cdot \log_2 n$	30	664	9,965	10^5	10^6	10^7
n^2	10^2	10^4	10^6	10^8	10^{10}	10^{12}
n^3	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
2^n	10^3	10^{30}	10^{301}	$10^{3,010}$	$10^{30,103}$	$10^{301,030}$

FACTORS THAT CONTRIBUTES TO RUNNING TIME OF ALGORITHM

- Single Processor / Multiprocessor
- Read/ Write to Memory
- 64 bit vs. 32 bit
- Input

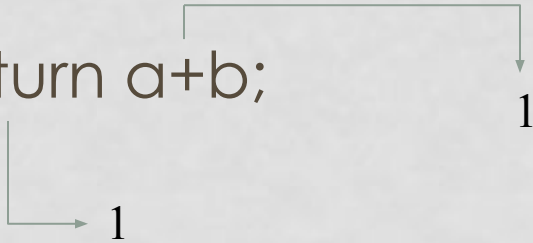
HOW TO ANALYZE TIME COMPLEXITY

- Model Machine
 - Single processor
 - 32 bit
 - Sequential Execution
 - Consumes
 - Arithmetic / logical Operations – 1 unit
 - Assignment Operations – 1 unit
 - Function call / return – 1 unit
 - RAM read / write – 1 unit

EXAMPLE 1

```
int sum (int a , int b)
```

```
{  
  return a+b;  
}
```



Tsum = 2

EXAMPLE 2

```
int sumOfArray (int arr[], int size)
{
  int sum=0;
  for(int i = 0 ; i < size; i++)
    sum=sum+arr[i];
  return sum;
}
```

Cost (unit)	iteration
1	1
2	n+1
3	n
1	1

$$\begin{aligned}T(n) &= 1 + 2n + 2 + 3n + 1 \\ &= 4 + 5n\end{aligned}$$

EXAMPLE 3

```
node* search(int item)
{
    node*t=head;
    while(t!=NULL)
    {if(t->val==item)
        return t;
      t=t->next
    }
    return t;
}
```

$$\begin{aligned} f(n) &= 1 + 1(n+1) + 2n + 2n + 1 \\ &= 1 + n + 1 + 2n + 1 + 2n = 3 + 5n = O(n) \end{aligned}$$

units	No. of times
1	1
1	n+1
2	n
1	1
2	n
1	1

EXAMPLE 4

```
bool search(int v)
{
  for (int i=0; i<size;i++)
  {
    if(arr[i]==v)
      return 1;
  }
  return 0;
}

f(n)=2(n+1)+2n+1
    =2n+2+2n+1=4n+3=O(n)
```

units	No. of times
2	n+1
2	n
1	1
1	1

ASYMPTOTIC NOTATIONS

- Function can be written in
 - Big-Oh Notation $O ()$ **Upper Bound**
 - Omega Notation $\Omega ()$ **Lower Bound**
 - Theta Notation $\phi ()$ **Average Bound**

For Complexity analysis, Time Complexity equation are converted into Big-Oh Notation

WHY NEED BIG-OH NOTATION

$$f(n)=4n+3$$

n	f(n)
10	43
10000	40,003
1000000	4,000,003
100000000	400,000,003
10000000000	40,000,000,003
1000000000000	4,000,000,000,003
10000000000000	40,000,000,000,003
100000000000000	4,000,000,000,000,003
1000000000000000	40,000,000,000,000,003
10000000000000000	400,000,000,000,000,003

BIG-OH NOTATION

Big-Oh($F(n)$) can be written as

- Take the highest order term
- Ignore all additive
- Ignore all Multiplicative

Big-Oh is also known as upper bound of a function written as $O ()$

MATHEMATICAL PROPERTIES OF BIG-O NOTATION

- You can ignore low-order terms in an algorithm's growth rate function. For example, if an algorithm is $O(N^3+4N^2+3N)$, then it is also $O(N^3)$
- You can ignore a multiplicative constant in the high-order term of an algorithm's growth rate function. For example, if an algorithm is $O(5N^3)$, then it is also $O(N^3)$
- $O(f(N))+O(g(N))=O(f(N)+g(N))$. For example if an algorithm is $O(N^2)+O(N)$, then it is also $O(N^2+N)$, which you can simply write as $O(N^2)$

SOME COMMON GROWTH RATES

- **Constant : $O(1)$**
 - Time Requirement is Constant
 - Independent of Problem Size
 - Example: Accessing an array element
- **Logarithmic : $O(\log N)$**
 - Time Requirement Increases **Slowly** for a Logarithmic Function
 - Typical algorithm: Solves a Problem by **Solving a Smaller Constant Fraction of the Problem**
 - **Example: Binary search**
- **Linear : $O(N)$**
 - Time Requirement Increases Directly with the size of the Problem
 - Example: Retrieving an element from a linked list

SOME COMMON GROWTH RATES

- $O(N \log N)$
 - Time Requirement Increases More Rapidly Than a Linear Algorithm
 - Typical algorithms : Divide a Problem into Smaller Problems That are **Each Solved Separately**
- Quadratic : $O(N^2)$
 - Increases Rapidly with the Size of the Problem
 - Typical Algorithms : Algorithms that Use **Two Nested Loops**
 - Practical for Only Small Problems
- Cubic : $O(N^3)$
 - Increases Rapidly with the Size of the Problem
 - Typical Algorithms : Algorithms that Use Three Nested Loops

SOME COMMON GROWTH RATES

- Exponential time: $O(2^N)$
 - **Very, very, very bad, even for small N**
 - **Usually Increases too Rapidly to be Practical**

CLASS EXERCISE

- $n^6 + 3n = O(?)$
- $2^n + 12 = O(?)$
- $3^n + 2^n = O(?)$
- $n^n + n = O(?)$

COMPLEXITY OF RECURSIVE PROGRAMS

```
factorial (n)
{
  if(n==0)
    return 1;
  else
    return n*factorial(n-1);
}
```

EXERCISE 1

```
int recursiveFun1 (int n)
{
    if (n <= 0)
        return 1;
    else return 1 + recursiveFun1 (n-1); }
```


EXERCISE 2

```
int recursiveFun2(int n)
{
    if (n <= 0)
        return 1;
    else return 1 + recursiveFun2(n-5);
}
```

EXERCISE 3

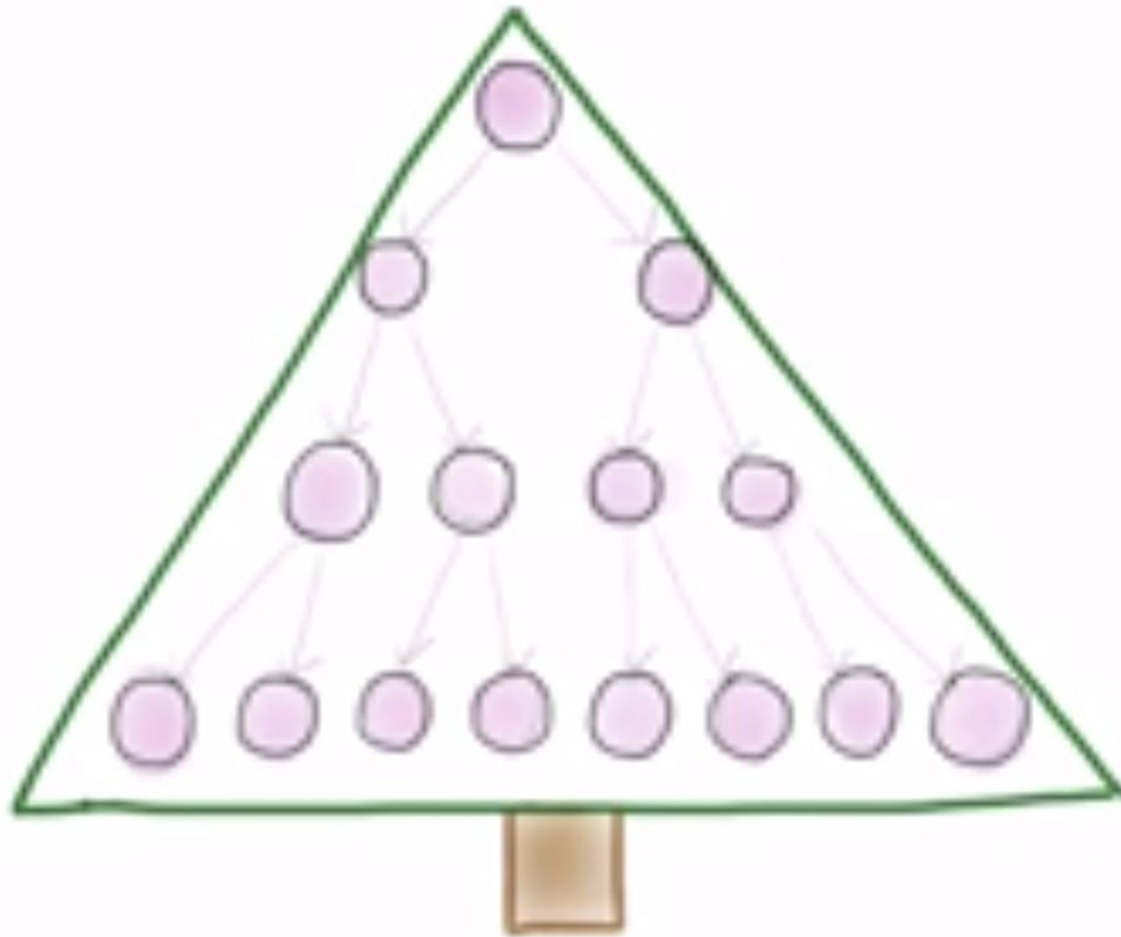
```
int recursiveFun3(int n)
{ if (n == 1)
  return 1;

  else
    return 1 + recursiveFun3(n/5); }
```

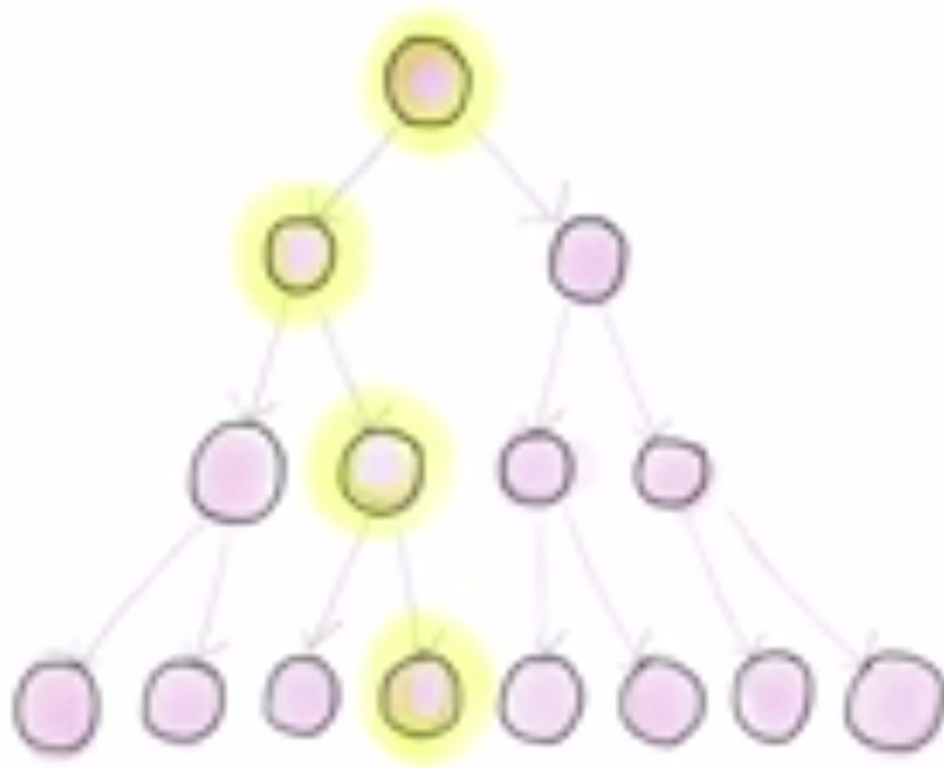
TIME COMPLEXITY OF BST

$\log_2$

PERFECT BINARY SEARCH TREE

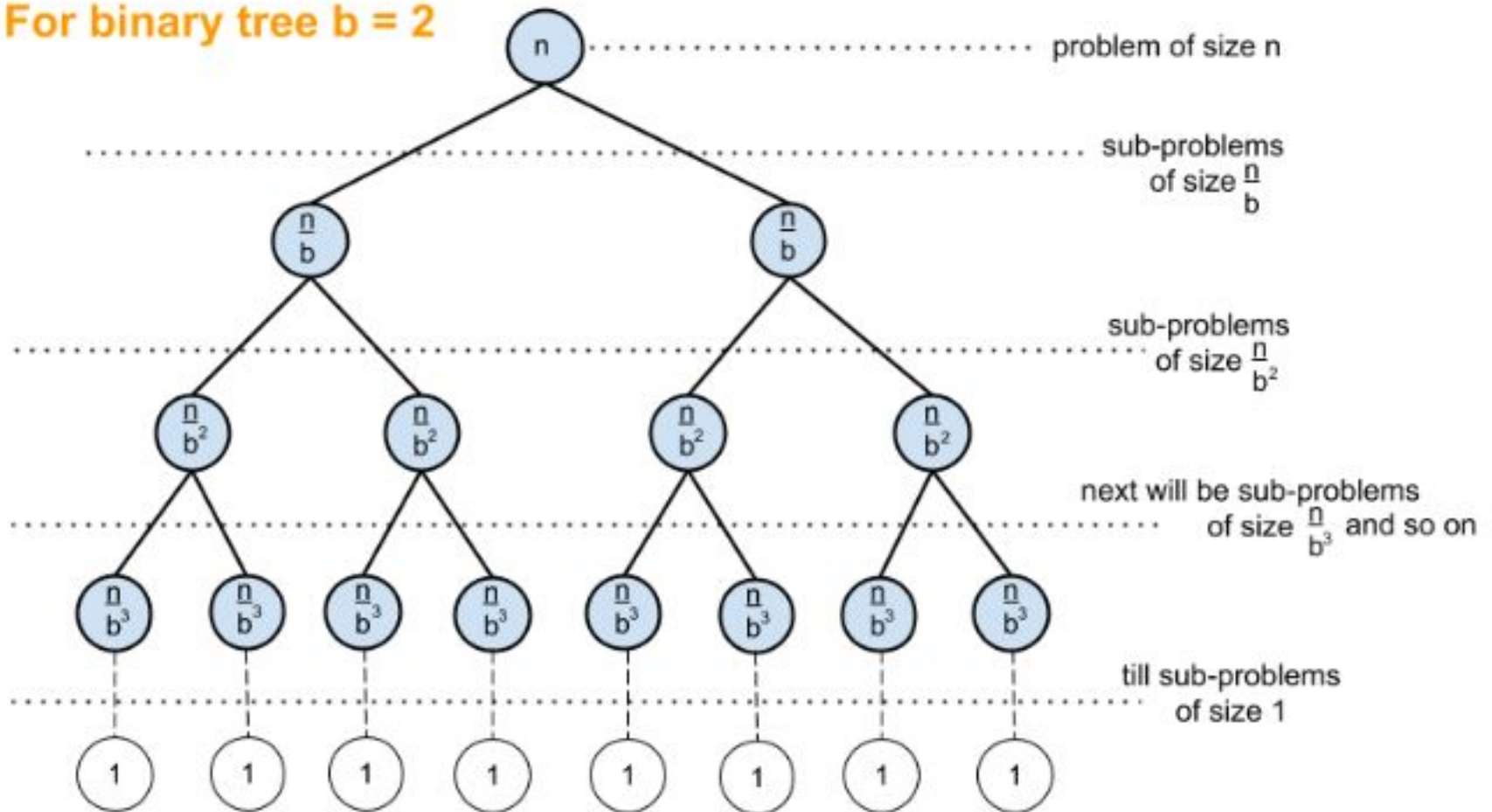


SEARCH A BST



COMPLEXITY OF BST

For binary tree $b = 2$



CONTD.

$$n * \left(\frac{1}{2}\right)^x = 1$$

CONTD.

$$\text{Log}_a n = x$$

READING

- Chapter 9, Data Abstraction and Problem Solving using C++, Walls & Mirrors 3rd ed.

HELP

- <https://www.youtube.com/watch?v=V42FBiohc6c&t=92s>
- <https://www.youtube.com/watch?v=8syQKTdgdzc&t=1s>
- <https://www.youtube.com/watch?v=ncpTxqK35PI>